

자연어 처리

(NATURAL LANGUAGE PROCESSING)

- 프로그램 내용:
 - 자연어처리 입문
 - Embedding의 이해
 - Transformers
 - GPT, BERT
- 프로그램 위치:
 - github.com/cshoontech/pytorch-nlp

기계에게 인간의 언어를 가르치는 첫걸음

자연어 처리의 뼈대, 토큰화(Tokenization)의 모든 것

인간의 언어는 구조화되지 않은 데이터(Unstructured Data)입니다.

인공지능이 이를 이해하고 분석하기 위해서는, 의미 있는 최소 단위로 쪼개는 정교한 과정이 필수적입니다.



여정의 이정표: 단순함에서 복잡함으로

6. 진정한 의미의 해독: 품사 태깅과 한국어 형태소 분석 (KoNLPy)

단순 토큰화를 넘어 각 단어의 문법적 기능 파악.
KoNLPy를 활용한 한국어 형태소 분석의 중요성.

5. 한국어, 자연어 처리의 '최종 보스': 교착어와 띄어쓰기의 난제

4. 문맥의 경계: 문장 토큰화의 딜레마 (NLTK vs KSS)

영어와 근본적으로 다른 한국어의 특성.
교착어의 복잡성과 띄어쓰기 불규칙성이 초래하는 어려움.

4. 문맥의 경계: 문장 토큰화의 딜레마 (NLTK vs KSS)

단순 제거 대상이 아닌 문맥 파악의 중요한 단서.
약어, 금액, 이메일 등 특수 구두점 처리의 중요성.

3. 예외의 연속: 구두점이 가진 숨겨진 의미들

단순 제거 대상이 아닌 문맥 파악의 중요한 단서.
아포스트로피(') 처리 방식의 차이를 통한 설계 철학 비교 분석.

2. 단어 토큰화의 함정: 도구별 설계 철학 (NLTK vs Keras)

동일한 입력에도 도구마다 다른 결과. 아포스트로피(')
처리 방식의 차이를 통한 설계 철학 비교 분석.

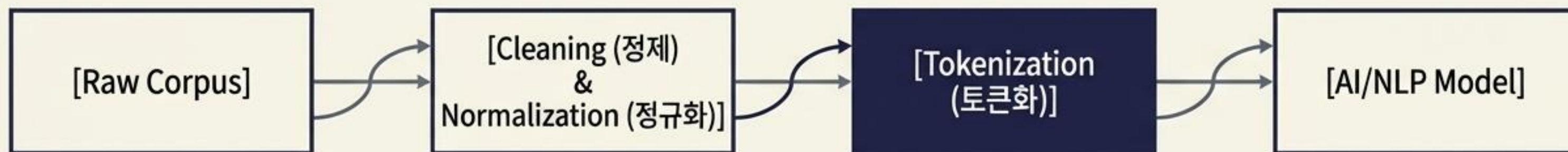
1. 토큰화의 정의와 필요성: 코퍼스에 의미 부여하기

텍스트 데이터를 의미 있는 단위로 분해하는 자연어 처리의 기초 단계.
기계가 이해할 수 있는 형태로 변환하는 필수적인 과정입니다.

무의미한 텍스트 덩어리에 의미를 부여하다

토큰화(Tokenization)란?

주어진 코퍼스(Corpus) 데이터에서 의미 있는 단위인 '토큰(Token)'으로 나누는 작업.



가장 순진한 접근법 (Naive Approach)

구두점을 단순히 제거하고 띄어쓰기(Whitespace)를 기준으로 자르면 어떻게 될까요?

[입력]: Time is an illusion. Lunchtime double so!

→

[출력]: "Time", "is", "an", "illustion", "Lunchtime", "double", "so"

단순히 공백으로 자르는 것, 과연 완벽할까요?

단어 토큰화 (Word Tokenization): 첫 번째 도구

NLTK 라이브러리를 활용한 기본적인 단어 단어 토큰화 실습.

주목할 포인트:

- Don't ➡ Do 와 n't 로 분리
- Jone's ➡ Jone 과 's 로 분리

NLTK Word Tokenizer Output

D2Coding

```
from nltk.tokenize import word_tokenize

text = "Don't be fooled by the dark sounding name, Mr.
Jone's Orphanage is as cheery as cheery goes for a pastry
shop."

print(word_tokenize(text))
```

```
['Do', 'n't', 'be', 'fooled', 'by', 'the', 'dark', 'sound', 'ing', 'name', '', 'Mr.', 'Jone', 's', 'Orphanage', 'is', 'as', 'cheery', 'as', 'cheery', 'goes', 'for', 'a', 'pastry', 'shop', '.']
```

목적이 도구를 결정한다: NLTK vs Keras

아포스트로피(')를 처리하는 방식은 도구의 설계 철학에 따라 완전히 다릅니다.

1. NLTK (WordPunctTokenizer)

구두점을 별도의 토큰으로 엄격하게 분류.

[입력]: Don't / Jone's

[출력]: ['Don', "'", 't'] /
['Jone', "'", 's']

2. Keras (text_to_word_sequence)

알파벳 소문자화 및 기본 구두점 제거, 하지만
아포스트로피는 보존.

[입력]: Don't / Jone's

[출력]: ["don't"] / ["jone's"]

단순한 정제(Cleaning)가 위험한 이유

구두점이나 특수문자를 단순히 제외하면 텍스트는 원래의 의미를 상실합니다.



단어 자체에 구두점이 포함된 경우:

m.p.h / Ph.D. / AT&T

특수 기호가 맥락을 결정하는 경우:

가격: \$45.55 (45와 55로 분리하면 안 됨)

날짜: 01/02/06

숫자 단위 표기:

123,456,789 (세 자리 단위 콤마)

접어(Clitic)와 합성어:

what're (what are), I'm (I am)

띄어쓰기가 포함된 단일 어휘: New York,

rock 'n' roll

규칙의 표준화: Penn Treebank Tokenization

가장 널리 쓰이는 표준 토큰화 알고리즘 중 하나.

핵심 규칙

- 규칙 1. 하이픈(-)으로 연결된 단어는 하나로 유지한다.
- 규칙 2. 아포스트로피로 '접어(clitic)'가 함께하는 단어는 분리한다.

[입력]: "Starting a home-based restaurant may be an ideal. it doesn't have a food chain..."

[출력]: [..., 'home-based', ..., 'does', "n't", ...]

home-based는 하나의 토큰으로 유지되고, doesn't는 분리된 것을 확인할 수 있습니다.

문맥의 경계를 찾아서: 문장 토큰화(Sentence Tokenization)

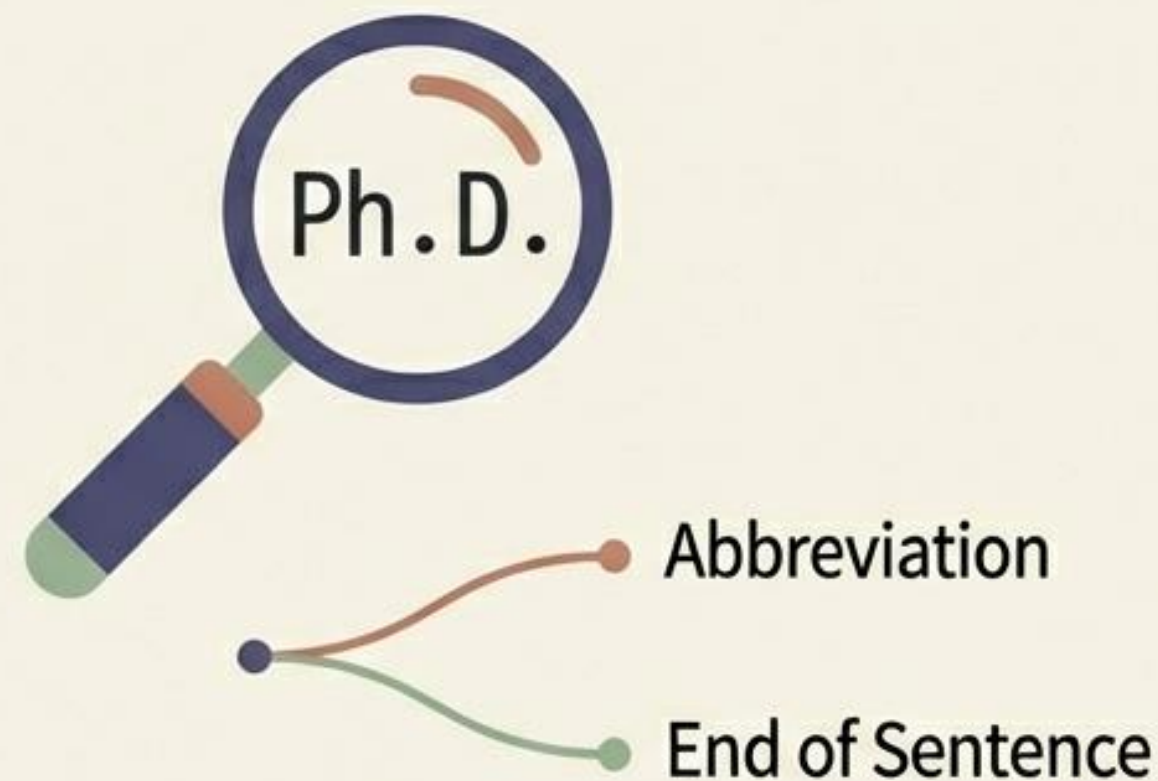
문장 분류(Sentence segmentation). 마침표(.)나 느낌표(!), 물음표(?)로 자르면 충분할까요?

마침표의 딜레마

EX1) IP 192.168.56.31 서버에 들어가서...

EX2) Since I'm actively looking for Ph.D. students, I get the same question...

단순히 마침표로 자른다면, 위 문장들은 중간에 완전히 끊어지고 맙니다.



[NLTK sent_tokenize 실습]:

[입력]: "I am actively looking for Ph.D. students. and you are a Ph.D student."

[출력]: ['I am actively looking for Ph.D. students.', 'and you are a Ph.D student.']

(NLTK는 Ph.D.를 단어로 인식하여 문장을 완벽하게 분리해냅니다.)

한국어 문장 토큰화: KSS (Korean Sentence Splitter)

한국어 문장 토큰화에는 박상길님이 개발한 KSS 라이브러리가 탁월한 성능을 보여줍니다.

Korean NLP Toolbox

```
import kss
text = '딥 러닝 자연어 처리가 재미있기는 합니다. 그런데 문제는 영어보다 한국어로 할 때 너무 어렵습니다. 이제 해보면 알걸요?'
print(kss.split_sentences(text))
```

```
['딥 러닝 자연어 처리가 재미있기는 합니다.',
 '그런데 문제는 영어보다 한국어로 할 때 너무 어렵습니다.',
 '이제 해보면 알걸요?']
```


한국어 토큰화의 난제 1: '교착어'의 특성

한국어는 영어처럼 띄어쓰기(어절) 단위로 토큰화를 하면 실패합니다. 한국어는 조사, 어미 등을 붙여서 말을 만드는 교착어이기 때문입니다.

- 조사의 무한 증식: '그' → 그가, 그에게, 그를, 그와, 그는 (기계는 이를 모두 다른 단어로 인식합니다.)

해결책: 형태소(Morpheme) 단위 토큰화 (가장 작은 말의 단위)



한국어 토큰화의 난제 2: 무너진 띄어쓰기

한국어는 띄어쓰기가 영어보다 덜 엄격하며, 지켜지지 않은 코퍼스(뉴스, 웹 데이터 등)가 매우 많습니다.

제가 이렇게 띄어쓰기를 전혀 하지 않고 글을 썼다고 하더라도 글을 이해할 수 있습니다.

EX1) (한국어: 띄어쓰기가 없어도 의미 파악이 가능하여 실데이터에 오류가 잦음)

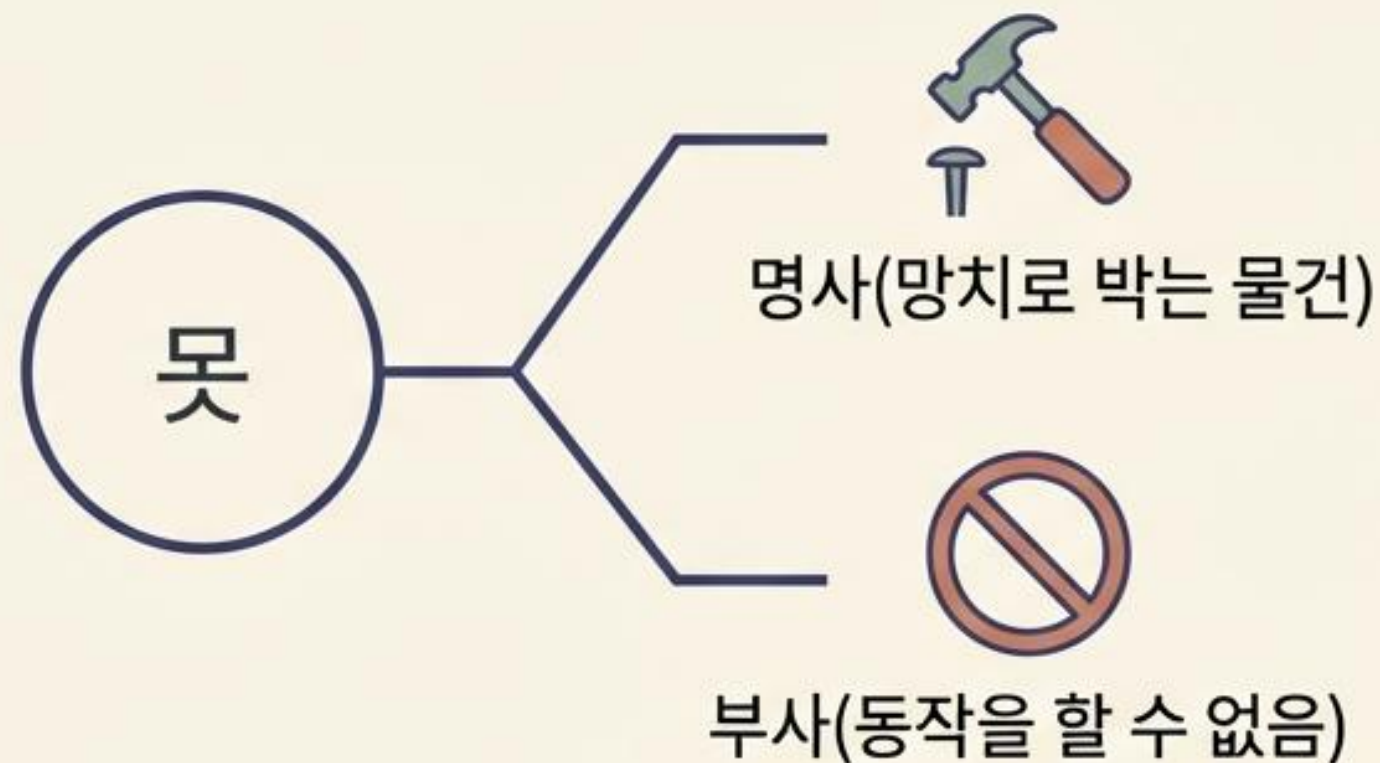
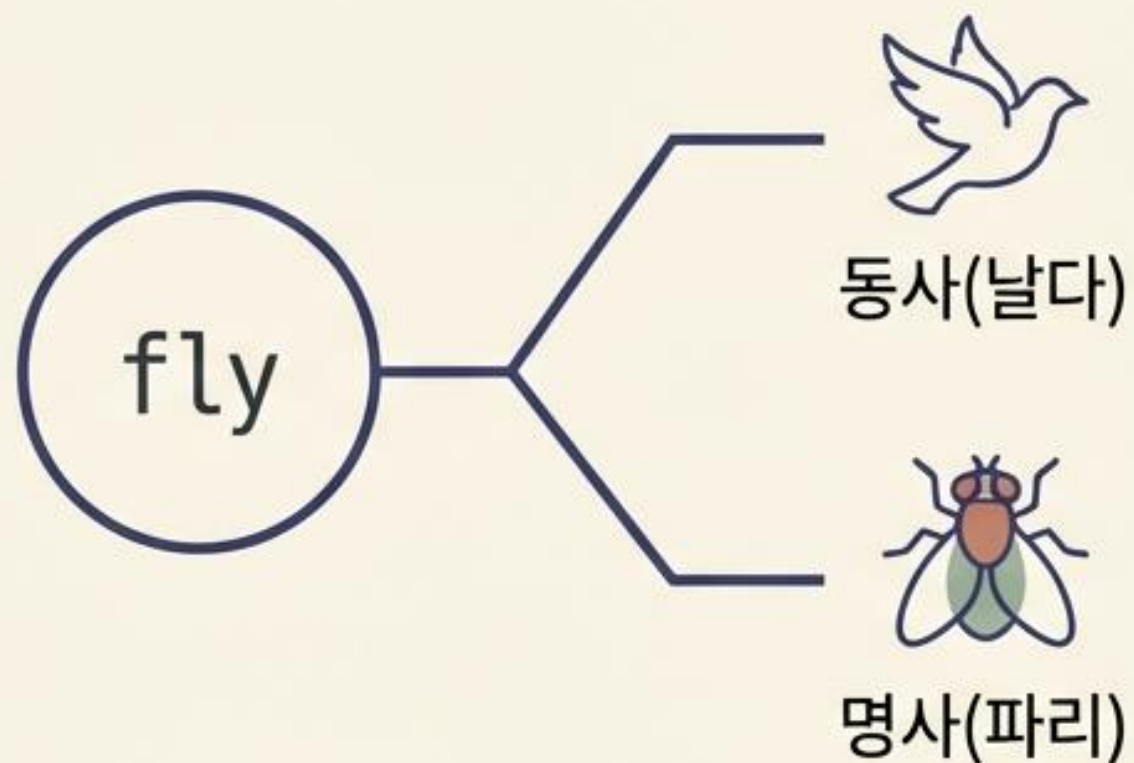
Tobeornottobetthatisthequestion

EX2) (영어: 띄어쓰기가 없으면 해독이 거의 불가능함)

결론적으로, 한국어 자연어 처리는 띄어쓰기에 의존할 수 없습니다.

진정한 의미의 해독: 품사 태깅 (POS Tagging)

단어의 표기는 같지만, 품사에 따라 의미는 완전히 달라집니다.



단어의 의미를 정확히 기계에게 전달하려면, 해당 단어가 어떤 품사로 쓰였는지 꼬리표를 달아주는 품사 태깅(Part-of-speech tagging)이 필수적입니다.

품사 태깅 실습: 영어 (NLTK)

NLTK는 Penn Treebank POS Tags 기준을 사용하여 품사를 분류합니다.

```
from nltk.tag import pos_tag
text = "I am actively looking for Ph.D. students. and you are a Ph.D. student."
tokenized = word_tokenize(text)
print(pos_tag(tokenized))
```

```
[('I', 'PRP'), ('am', 'VBP'), ('actively', 'RB'), ('looking', 'VBG'), ('for', 'IN'), ('Ph.D.', 'NNP'), ('students', 'NNS'), ('.', '.'), ('and', 'CC'), ...]
```

I	am	actively	looking	for	Ph.D.,	students	.	and
PRP	VBP	RB	VBG	IN	NNP	NNS	.	CC

태그 의미 : PRP(대명사), VBP(동사), RB(부사), NNP(고유명사)

한국어 형태소 분석기 마스터클래스: KoNLPy

한국어 자연어 처리를 위한 파이썬 패키지 KoNLPy. 목적과 속도에 따라 형태소 분석기를 선택해야 합니다.

[입력]: "열심히 코딩한 당신, 연휴에는 여행을 가봐요"

1. Okt (Open Korea Text)

형태소(morphs): ['열심히', '코딩', '한', '당신', '', '연휴', '에는', '여행', '을', '가봐요']

품사(pos): ('코딩', 'Noun'), ('한', 'Josa'), ('가봐요', 'Verb')

2. 꼬꼬마 (Kkma) (더 세밀한 분석)

형태소(morphs): ['열심히', '코딩', '하', 'ㄴ', '당신', '', '연휴', '에', '는', '여행', '을', '가보', '아요']

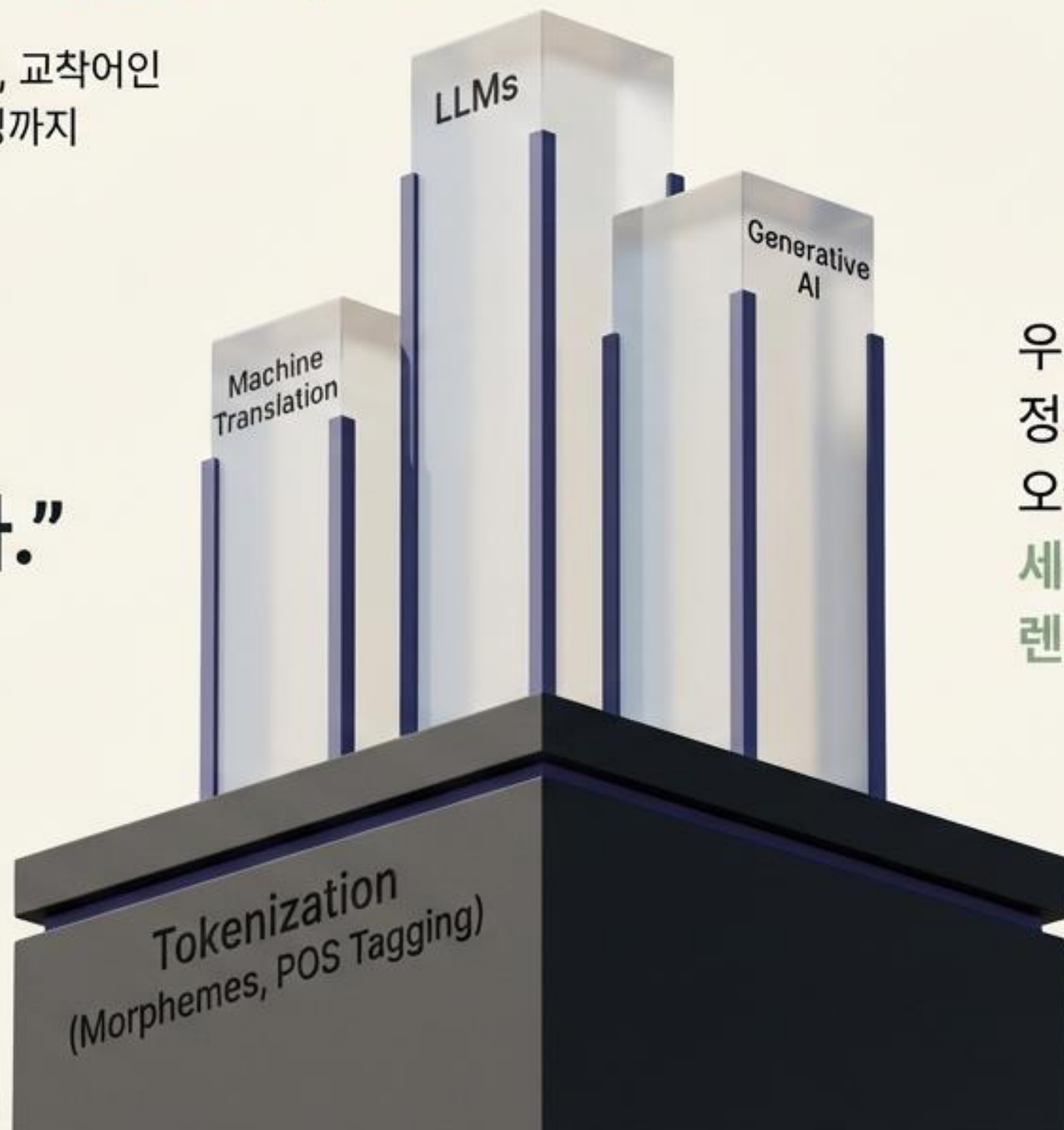
품사(pos): ('코딩', 'NNG'), ('하', 'XSV'), ('ㄴ', 'ETD')

(Okt는 '한'을 하나의 조사로 보지만, 꼬꼬마는 '하'와 'ㄴ'으로 더 잘게 쪼갭니다.)

텍스트에서 AI로: **토큰화**는 모든 언어 모델의 **기반**입니다

우리는 오늘 단순한 띄어쓰기 자르기부터, 교착어인
한국어의 복잡한 형태소 분석과 품사 태깅까지
살펴보았습니다.

“기계는 스스로
글을 읽지 못합니다.”



우리가 설계한 규칙과 의도에 따라
정제되고 **토큰화된 데이터**만이,
오늘날 거대한 언어 모델(LLM)이
**세상을 이해하는 가장 완벽한
렌즈**가 됩니다.